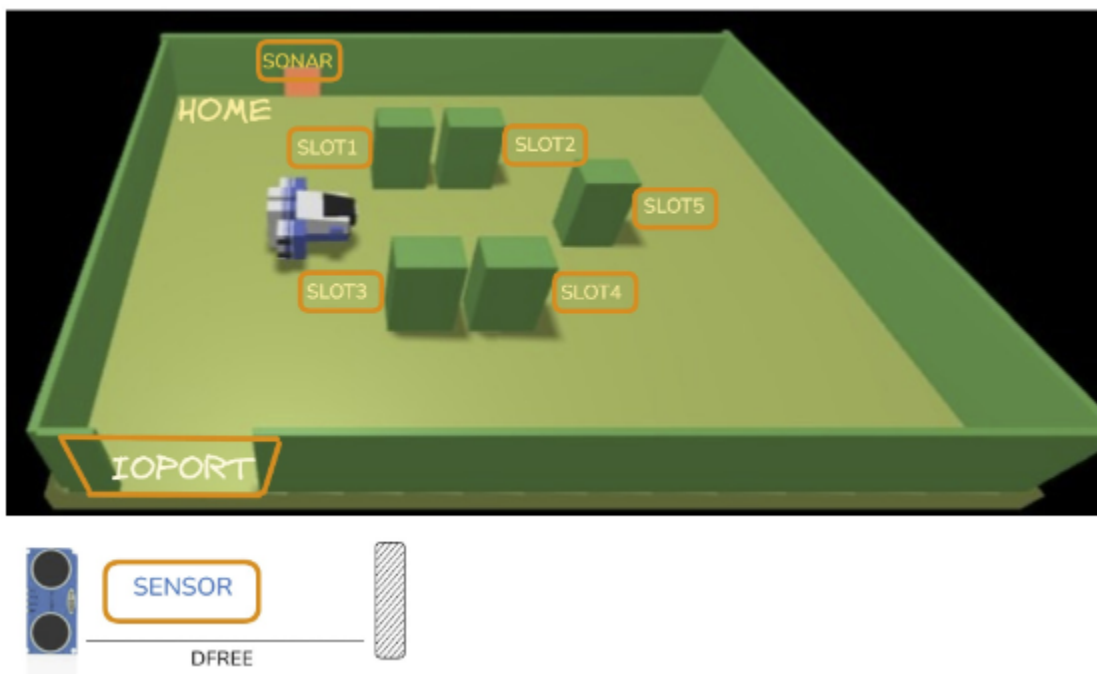


Sprint 1

Marchini Claudio, Tomasi Cesare

Introduction

A Maritime Cargo shipping company (from now on, simply **company**) intends to automate the operations of load of **containers** in the ship's cargo hold (or simply hold). To this end, the company plans to employ a *Differential Drive Robot* (from now, called **cargorobot**). The *hold* is a rectangular, flat area with an Input/Output port (**IOPort**). The area provides 4 slots to store the containers and a slot named **slot5**.



In the picture above:

- The **slots1-4** depict the hold areas reserved to store one *container* each,
- The **slots5** depicts an area, where the *cargorobot* must temporarily store a

container, before to place it in one of the *slots1-4*. During temporary storage, a 'marker' device labels the container with an identification barcode and signals when this marking activity is completed.

- The **IOPort** is a device with a **pushbutton** and a **display**. The *pushbutton* is pressed by the customer in order to send a request to load a container on the cargo. The *display* is used to show the answer to the request and to show the current state of the *hold*.
- The **sensor** associated to the *IOPort* is a device (a *sonar*) used to detect the presence of a container, when it measures a distance D , such that $D < D_{FREE}/2$, during a reasonable time (e.g. 3 secs).

Requirements

The company asks us to build service named **cargoservice** that should work as follows. The *cargoservice* is able to receive a **request to load** a container sent by some customer by using the *pushbutton* of the *IOPort*.

- It sends the answer *retrylater*, if the **IOPort** is currently occupied by a container or if the system is *Out of service*
- It rejects the request when the hold is already full, i.e. the *slots1-4* are already occupied.
- Otherwise, it considers the system as *engaged*, detects a free slot and returns as answer the name of such a reserved slot. While engaged, the system must blink a Led.

When the *request to load* is accepted, the customer must move the container in the *sensor* area within prefixed amount of time (e.g. 30 secs), otherwise the systems becomes **disengaged**. Then, the *cargoservice* uses the *cargorobot* to move the container from the *IOPort* to the *slot5* (for marking the container) and then to the reserved slot. The service must also show on the *display* on the *IOPort*:

- the current state of the *hold*
- the message '**Service working**', when it is all is going well

- the message '**Out of service**' if the *sonar sensor* measures a distance $D > D_{FREE}$ for at least 3 secs (perhaps a failure of the *sonar*).

Requirement Analysis

This requirement analysis builds on top of what is defined in the sprint 0. The main objective of this sprint's requirement analysis is to define the *marker device*.

In order to implement the **robotplanner** defined in the previous sprint, we need to define the marker device: from the requirements we can deduce that the *marker* will need to communicate with the rest of the system in order to signal when it has completed its job and possibly specify the barcode assigned to the container.

Problem Analysis

The system need an actor that manages the whole flow of the application, we will call this actor **cargoservice** and will work as the main unit, which will dispatch jobs to other helper services. The normal flow defined in the sprint 0 is:

1. It listens for a **request to load** received by the *pushbutton*
2. It checks the state of the **IOPort**, and decides which answer to send back, as defined in the sprint 0
3. If a slot is free, it waits for the container to be placed in the *sensor area* and then delegates the *cargorobot* service to move the container to **slot-5**
4. It then moves the container from **slot-5** to the reserved slot again with the use of the **cargorobot**

Here we will solidify the definition of the behaviour by expanding and formalizing it using *qak*:

```

QActor cargospace context ctxcargospace {
  [# val TimeoutMillis = 30000 #]
  [# val ReservedSlot = 0 #]

  State s0 disengaged {
    // Wait for a request to load

    if [# ioport occupied #] {
      //send retrylater
    } else if [# slots occupied #] {
      //send rejected
    } else {
      //reserve slot
      //set reserved slot var
      //send accepted(slot id)
      Goto engaged
    }
  }

  State engaged {
    //call blink led service
  }

  Transition t0
    whenTime TimeoutMillis -> disengaged
    whenMessage IOPortDeposited -> moveRobot

  State moveRobot {
    // call robot service from IOPort to slot 5
    // call robot service mark container
    // call robot service from slot 5 to slot ReservedSlot

    //stop blink led service
    Goto disengaged
  }
}

```

The **sonar** is a HC-SR04 and it is attached to a Raspberry Pi Pico W. The device will need to communicate with the rest of the system but in order to define *how* we will communicate we need to consider a few limitations of the hardware:

- The raspberry is too lightweight to support a full JVM needed to run *qak* directly,

so we have to use either cpp or micropython as the language

- Since *qak* is not a possibility the communication must be implemented with a protocol, preferably one that *qak* supports out of the box

```
def connectWifi():
    #connect to wifi

    print("Connected:", wlan.ifconfig())

def connectMqtt():
    # create Mqtt client
    # connect to Mqtt broker
    return client

def measureDistance():
    # measure distance using sonar
    return distance

TRIG = Pin(3, Pin.OUT)
ECHO = Pin(2, Pin.IN)

connectWifi()
client = connectMqtt()

while True:
    try:
        d = measureDistance()
        if d is not None:
            # create formatted msg
            client.publish(TOPIC, msg.encode() )

            time.sleep(1)
    except Exception as e:
        print("Exception: ", type(e).__name__, e)
```

- We have assumed that the Board will be connected through Wifi
- The configurations parameters (Wifi SSID, Password, MQTT broker, topic, ect..) will be set in a *.env* file in order to enhance reusability.
- The *msg* will be formatted as specified by the documentation of [unibo.basicomm23-1.0](#), a library developed by our software house, already used

by *qak*

Once we have a way to receive distance information from the raspberry we have two main options going forward:

- Forward the raw distance to the rest of the system, it the will be the responsibility of each component to interpret the information and evaluate the correct state (sonar working / out of service)
- Add an intermediate component that takes the raw distance, processes it, and emits only the relevant messages (sonar working / out of service) to the rest of the system

```
Event serviceWorking : serviceWorking(X)
```

```
Event outOfService : outOfService(X)
```

```
Dispatch containerInIoPort : containerInIoPort(X)
```

```
QActor sonarwrapper context ctxcargoservice {
```

```
    State s0 initial {
```

```
        // listen to the sonar messages and send the right events/dispatch
```

```
    }
```

```
}
```

Since the **IOPort** has to maintain the current status of the system at any moment the web page will need a **WebSocket** connection to ensure it receives updates, also receive the state of the *hold* our implementation in the requirement analysis will also need a listener that will forward any change via WebSocket to all connected clients.

```

const socket = new WebSocket("ws://localhost:8080");

const requestSpan = document.getElementById("request-result");
const statusSpan = document.getElementById("current-status");

socket.addEventListener("open", (event) => {
  //Request initial state
});

socket.addEventListener("message", (event) => {
  // Dispatch by message type and update right span
});

```

The system also needs to wait for a **marker** device to signal the end of its activity. In the previous analysis we did not specify if this component is given by the client or we have to build it by ourselves. For now we will implement a mock QActor

```

Request markContainer : markContainer(ID)
Reply markDone : markDone(msg) for markContainer

QActor marker : context ctxcargoservice {
  [# val MarkingTime = 5000 #]

  State s0 initial {

  }
  Transition t0
    whenMessage markContainer -> handleMarkRequest

  State handleMarkRequest {
    //Wait for marking and send reply
  }
  Goto s0
}

```

This design maintains a clear separation between the core logic (QActor) and the rest of the system, this enables the project to quickly adapt to whatever the client will need (either a custom marker or one built by us) by just adding a QActor that sends/ receives the specified messages.

Test Plans

Project

Testing

Deployment

Maintenance



Marchini Claudio

claudio.marchini@studio.unibo.it



Tomasi Cesare

cesare.tomasi@studio.unibo.it



SignedSnow0/CargoService